HashMath: A Content-Addressed Calculus of Inductive Constructions for Permissionless Formal Mathematics

Samuel Schlesinger

Abstract

Formalized mathematics is fragmented across incompatible, name-dependent systems. Lean, Rocq, and Isabelle each maintain curated libraries where every definition and theorem is identified by a human-chosen name, creating coordination bottlenecks that limit the scale at which humans and AI can contribute. We observe that mathematical identity is structural, not nominal: two proof terms with identical structure are the same proof regardless of what anyone calls them. We propose **HashMath**, a content-addressed Calculus of Inductive Constructions (CIC) in which every well-typed term is indexed by its SHA-256 hash, with all dependencies referenced by hash rather than by name. The result is a global, permissionless, append-only knowledge base where correctness is guaranteed by type theory, names are optional metadata, and contributions require no coordination, review, or naming consensus. A type-to-inhabitants index enables type-directed search across the entire knowledge base, allowing AI agents and human mathematicians to discover relevant lemmas by their type signatures alone. HashMath provides the infrastructure for Tao's vision of collaborative human-AI mathematics at scale: many agents and mathematicians contributing simultaneously, building on each other's work by hash, without a single naming conflict.

1 Introduction

Formalized mathematics is at an inflection point. Mathlib, the community-driven library for Lean 4, now contains over 210,000 theorems and continues to grow at an accelerating pace [2]. Google DeepMind's AlphaProof achieved silver-medal performance at the 2024 International Mathematical Olympiad [3], and DeepSeek-Prover-V2 has reached gold-level IMO performance [4]. Terence Tao envisions a near future in which "big theorems are proved by 20 people and multiple AIs, each proving little things" [5].

Yet the infrastructure for this vision does not exist. Today's proof assistants—Lean [1], Rocq (formerly Coq), Agda, Isabelle/HOL—organize knowledge through *named references* within *curated repositories*. Every definition, lemma, and theorem must be given a unique name, reviewed by maintainers, and placed within a coherent namespace hierarchy. This works when a small community curates a single library, but it fundamentally limits scale.

The coordination bottleneck. Contributing to Mathlib requires conforming to naming conventions, passing linters, and waiting for human review. These are reasonable quality controls, but they create a throughput ceiling. When AI systems can generate thousands of correct proofs per hour, the review queue becomes the bottleneck, not the proving. Independent formalizations cannot compose without reconciliation of names, import paths, and stylistic conventions.

The naming problem. Mathematicians disagree on names. Is it `Nat.add_comm` or `plus_comm` or `addition_commutative`? Different communities, different languages, and different traditions choose differently. AI agents, which operate at machine speed, generate names like `theorem_7a3f`—functional but meaningless to humans. When two independent groups formalize the same theorem under different names, reconciliation requires human labor to identify the duplication, choose a canonical name, and redirect references.

The Polymath precedent. In 2009, Timothy Gowers launched the Polymath project, demonstrating that permissionless mathematical collaboration works [6]. Over 40 contributors proved the density Hales-Jewett theorem through blog comments, with the result published in the *Annals of Mathematics* [7]. But the coordination was ad hoc—blog comments, manual tracking, human synthesis. HashMath provides the infrastructure that Polymath lacked: a system where contributions compose automatically, correctness is verified mechanically, and no central coordinator is required.

Core insight. Mathematical objects have *intrinsic identity* determined by their structure. The proof that $n + 0 = n$ by induction on n is the same proof regardless of whether it is called `Nat.add_zero`, `plus_n_O`, or `h_7f3a2b`. Names are metadata—useful for human navigation but irrelevant to mathematical content.

We propose **HashMath**: a content-addressed Calculus of Inductive Constructions in which every well-typed term is identified by its cryptographic hash. Dependencies are referenced by hash, not by name. The result is a global, append-only knowledge base that requires no coordination to use, no curator to maintain, and no naming consensus to function.

> *Inspired by Bitcoin's insight that trust can emerge from verification rather than authority [24], HashMath applies content-addressing to formal mathematics: naming conflicts are resolved without trusted curators.*

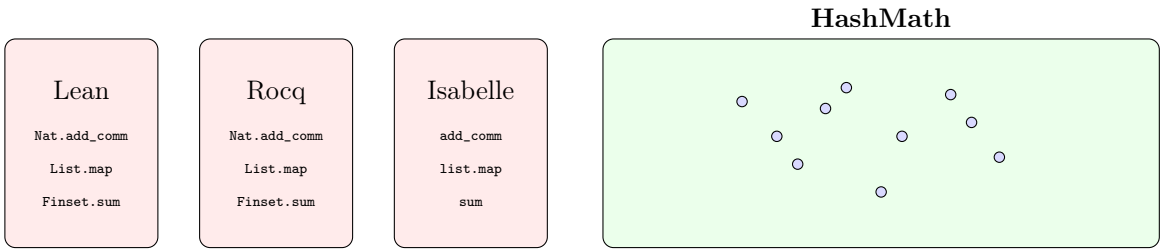
Paper outline. Section 2 specifies the HashMath CIC dialect—term constructors, typing rules, and reductions. Section 3 defines the content-addressing scheme and hash functions. Section 4 analyzes verification and trust properties. Section 5 describes the knowledge base architecture, including the type-to-inhabitants index and interoperability with existing proof assistants. Section 6 discusses AI and human collaboration at scale. Section 7 addresses challenges and limitations. Section 8 surveys related work. Section 9 describes the reference implementation's distribution prototype. Section 10 presents a phased development roadmap, and Section 11 outlines future work on advanced indices.

2 The Calculus of Inductive Constructions

This section gives a precise specification of the *HashMath CIC dialect*—the subset and variant of the Calculus of Inductive Constructions that HashMath typechecks and stores. The dialect is closely aligned with the Lean 4 kernel [1] but strips away all elaboration-only information (binder names, binder info, free variables, metavariables, literals, and metadata nodes) so that the core calculus is fully determined by the mathematical content. Our presentation follows Coquand and Huet [8], Coquand and Paulin-Mohring [9], and Werner [10], adapted to the content-addressed setting.

2.1 Term Constructors

Lean 4's kernel defines `Expr` with 12 constructors. HashMath retains 8 of these (those that carry type-theoretic content) and excludes the 4 that serve only elaboration, metaprogramming, or



(a) Fragmented silos: duplicated work, incompatible names

(b) Unified pool: one hash per proof

Figure 1: Formalized mathematics today exists in isolated silos with duplicated effort and incompatible naming. HashMath provides a single content-addressed pool where every unique proof term has exactly one identity.

syntactic sugar. Two new constructors are added: iref for block-relative references in mutual inductive definitions, and href for hash-referenced subterms in the storage layer (Section 3.7), for 10 constructors total.

Definition 2.1 (HashMath expression grammar)**.** HashMath terms are defined by the following grammar, parameterized by a type α:

$$
\begin{aligned}
e \quad ::= \quad & \mathsf{bvar}(n) \\
| \quad & \mathsf{sort}(u) \\
| \quad & \mathsf{const}(h, \bar{u}) \\
| \quad & \mathsf{app}(e_1, e_2) \\
| \quad & \mathsf{lam}(e_T, e_b) \\
| \quad & \mathsf{forallE}(e_T, e_b) \\
| \quad & \mathsf{letE}(e_T, e_v, e_b) \\
| \quad & \mathsf{proj}(h, i, e) \\
| \quad & \mathsf{iref}(k, \bar{u}) \\
| \quad & \mathsf{href}(a)
\end{aligned}
$$

where $n \in \mathbb{N}$, u is a universe level (Section 2.2), $h \in \{0,1\}^{256}$ is a content hash, $\bar{u}$ is a list of universe levels, $i \in \mathbb{N}$, $k \in \mathbb{N}$ is a block-relative type index, and $a : \alpha$ is a hash reference value whose type is determined by the phantom parameter α.

When $\alpha = \bot$ (the empty type), the href constructor is uninhabitable, guaranteeing at the type level that kernel expressions contain no hash references. When $\alpha = \mathsf{Hash}$ (32-byte SHA-256 digests), href carries a content hash pointing to a stored subterm. This phantom-type discipline ensures that hash references can never reach the type checker.

The constructors are:

- $\mathsf{bvar}(n)$ — a de Bruijn bound variable with index n.
- $\mathsf{sort}(u)$ — a universe sort at level u. This is the AST constructor; the familiar names Prop and $\mathsf{Type}(n)$ are abbreviations for $\mathsf{sort}(\mathsf{zero})$ and $\mathsf{sort}(\mathsf{succ}(n))$, respectively.
- $\mathsf{const}(h, \bar{u})$ — a reference to a previously declared constant identified by its content hash h, instantiated at universe levels $\bar{u}$. Crucially, constants are referenced *by hash*, not by name.
- $\mathsf{app}(e_1, e_2)$ — function application.

Lean 4 Constructor	Purpose	HashMath	Notes
`bvar`	Bound variable	✓	De Bruijn index
`fvar`	Free variable	—	Only closed terms stored
`mvar`	Metavariable	—	Elaboration only
`sort`	Universe sort	✓	
`const`	Named constant	✓	By hash, not name
`app`	Application	✓	
`lam`	Lambda	✓	Name + info excluded
`forallE`	Dependent product	✓	Name + info excluded
`letE`	Let binding	✓	Name excluded
`lit`	Literal	—	Expand to constructors
`mdata`	Metadata	—	Elaboration only
`proj`	Projection	✓	
—	Block-relative ref	✓	`iref`: resolved to `const`
—	Hash reference	✓	`href`: storage only (α=$\perp$ blocks kernel use)

Table 1: Lean 4 `Expr` constructors and their status in HashMath. Of the 12 Lean constructors, 8 are retained and 4 are excluded (elaboration or syntactic sugar). Two new constructors (`iref` and `href`) are added, for 10 total. The `href` constructor is parameterized by a phantom type, making it uninhabitable in kernel terms.

- $\mathsf{lam}(e_T, e_b)$ — lambda abstraction with domain type e_T and body e_b. Binder name and binder info (implicit/explicit/instance) are *excluded*: they are elaboration metadata, not type-theoretic content.
- $\mathsf{forallE}(e_T, e_b)$ — dependent product Π-type with domain e_T and codomain e_b. Binder name and binder info are again excluded.
- $\mathsf{letE}(e_T, e_v, e_b)$ — let binding with type annotation e_T, value e_v, and body e_b. Binder name is excluded.
- $\mathsf{proj}(h, i, e)$ — projection of the i-th field from a structure whose inductive type has hash h, applied to term e.
- $\mathsf{iref}(k, \bar{u})$ — a block-relative reference to the k-th type within a mutual inductive block, instantiated at universe levels $\bar{u}$. Used inside constructor types before the block hash is known; resolved to const when the block is registered (Section 3.4).
- $\mathsf{href}(a)$ — a hash reference to a stored subterm. Present only in the storage/wire format ($\alpha = \mathsf{Hash}$); absent from kernel terms ($\alpha = \perp$). See Section 3.7.

Excluded constructors. Table 1 summarizes the status of all 12 Lean 4 `Expr` constructors. The four excluded constructors are:

- fvar (free variables) — HashMath stores only *closed* terms; all variables under consideration are bound.
- mvar (metavariables) — placeholders for the elaborator; absent from fully elaborated kernel terms.
- lit (literals) — Lean 4 uses lit for efficient `Nat` and `String` literals. These are syntactic sugar that can always be expanded to the corresponding constructor applications ($\mathsf{succ}(\mathsf{succ}(\cdots))$ for natural numbers, a list of characters for strings). HashMath requires the expanded form.
- mdata (metadata) — annotation nodes carrying elaboration metadata. These have no type-theoretic content and are stripped.

2.2 Universe Levels

Universe levels govern the stratification of types.

Definition 2.2 (Level)**.** The type of universe levels is defined by:

$$u \quad ::= \quad \mathsf{zero} \mid \mathsf{succ}(u) \mid \max(u_1, u_2) \mid \mathsf{imax}(u_1, u_2) \mid \mathsf{param}(n)$$

where $n \in \mathbb{N}$ is a de Bruijn index into the list of universe parameters of the enclosing declaration.

The constructors are:

- zero — the base level.
- $\mathsf{succ}(u)$ — the successor of level u.
- $\max(u_1, u_2)$ — the maximum of two levels.
- $\mathsf{imax}(u_1, u_2)$ — the *impredicative maximum*: equals zero if u_2 evaluates to zero, and $\max(u_1, u_2)$ otherwise. This is essential for typing Π-types into Prop (see Section 2.3).
- $\mathsf{param}(n)$ — a universe parameter variable, referenced by de Bruijn index rather than by name, for the same content-addressing reasons that binder names are excluded from terms.

Excluded constructor. Lean 4's `Level` type also has a `mvar` constructor for universe metavariables. These are elaboration-time placeholders and are excluded from HashMath, just as expression metavariables are excluded.

2.3 Typing Rules

A context Γ is a list of typed variable declarations. The judgment $\Gamma \vdash e : T$ asserts that term e has type T in context Γ.

$$\frac{}{\Gamma \vdash \mathsf{sort}(u) : \mathsf{sort}(\mathsf{succ}(u))}\ \textsc{Sort} \qquad \frac{\Gamma(n) = T}{\Gamma \vdash \mathsf{bvar}(n) : T}\ \textsc{Var} \qquad \frac{\Gamma \vdash A : \mathsf{sort}(i) \qquad \Gamma, A \vdash B : \mathsf{sort}(j)}{\Gamma \vdash \mathsf{forallE}(A, B) : \mathsf{sort}(\mathsf{imax}(i, j))}\ \textsc{Pi}$$

$$\frac{\Gamma \vdash A : \mathsf{sort}(i) \qquad \Gamma, A \vdash t : B}{\Gamma \vdash \mathsf{lam}(A, t) : \mathsf{forallE}(A, B)}\ \textsc{Lam} \qquad \frac{\Gamma \vdash f : \mathsf{forallE}(A, B) \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathsf{app}(f, a) : B[a/0]}\ \textsc{App}$$

$$\frac{\Gamma \vdash T : \mathsf{sort}(u) \qquad \Gamma \vdash v : T \qquad \Gamma, T \vdash b : B}{\Gamma \vdash \mathsf{letE}(T, v, b) : B[v/0]}\ \textsc{Let}$$

$$\frac{\mathsf{decl}(h) = (k, \tau) \qquad |\bar{u}| = k}{\Gamma \vdash \mathsf{const}(h, \bar{u}) : \tau[\bar{u}]}\ \textsc{Const} \qquad \frac{\Gamma \vdash e : T \qquad T \equiv T'}{\Gamma \vdash e : T'}\ \textsc{Conv}$$

Several points deserve emphasis:

Sort rule for Π-types. The sort of $\mathsf{forallE}(A, B)$ uses $\mathsf{imax}(i, j)$ rather than $\max(i, j)$. When $B : \mathsf{Prop}$ (i.e., $j = 0$), the product type also lives in Prop regardless of the level of A. This is what makes Prop impredicative: one can quantify over all types and still land in Prop.

Cumulativity. The universe hierarchy is cumulative:

$$\mathsf{Prop} \leq \mathsf{Type}(0) \leq \mathsf{Type}(1) \leq \mathsf{Type}(2) \leq \cdots$$

If $\Gamma \vdash e : \mathsf{sort}(u)$ and $u \leq u'$, then $\Gamma \vdash e : \mathsf{sort}(u')$. Cumulativity is built into the conversion rule: the definitional equality check $T \equiv T'$ accounts for subtyping of sorts.

De Bruijn conventions. Since binder names are absent, contexts are lists rather than named mappings, and substitution $B[a/0]$ replaces de Bruijn index 0 and shifts remaining indices. The notation Γ, A extends the context with a new variable of type A at index 0.

2.4 Reductions

Definitional equality $\equiv$ is generated by the following reduction rules:

- **β-reduction**: $\mathsf{app}(\mathsf{lam}(A, t),\ a) \ \longrightarrow_\beta\ t[a/0]$
- **δ-reduction**: unfold a constant to its definition. If $\mathsf{decl}(h)$ is a definition with value v, then $\mathsf{const}(h, \bar{u}) \ \longrightarrow_\delta\ v[\bar{u}]$. In HashMath, *all definitions are always transparent*: there is no opacity mechanism (see Section 2.10).
- **ι-reduction**: a recursor applied to a constructor computes. If rec_I is the recursor for inductive type I and c_k is the k-th constructor, then $\mathsf{rec}_I\ \bar{p}\ \bar{m}\ (c_k\ \bar{p}\ \bar{a}) \ \longrightarrow_\iota\ m_k\ \bar{a}$ (where $\bar{p}$ are parameters, $\bar{m}$ are motives and minor premises, and m_k is the minor premise for c_k).
- **ζ-reduction**: substitute let bindings. $\mathsf{letE}(T, v, b) \ \longrightarrow_\zeta\ b[v/0]$
- **Projection reduction**: $\mathsf{proj}(h_I, i, c_k\ \bar{p}\ \bar{a}) \ \longrightarrow\ a_i$, where c_k is the single constructor of the structure I and a_i is the i-th field argument.
- **Quotient reduction**: The Quot type (Section 2.8) has two computation rules:

$$\mathsf{Quot.lift}\ f\ h\ (\mathsf{Quot.mk}\ r\ a) \ \longrightarrow\ f\ a$$
$$\mathsf{Quot.ind}\ \varphi\ h\ (\mathsf{Quot.mk}\ r\ a) \ \longrightarrow\ \varphi\ a$$

In addition to these reduction rules, definitional equality includes the following equivalences:

- **η-equivalence**: $f \equiv \mathsf{lam}(A, \mathsf{app}(f', \mathsf{bvar}(0)))$ where f' is f with de Bruijn indices shifted up by 1, provided $f : \mathsf{forallE}(A, B)$.
- **Proof irrelevance for Prop**: if $p, q : T$ and $T : \mathsf{Prop}$, then $p \equiv q$. All proofs of a proposition are definitionally equal.
- **Structural η for structures**: if I is a structure (a single-constructor inductive with no indices) and $s : I\ \bar{p}$, then $s \equiv c(\bar{p}, \mathsf{proj}(h_I, 0, s), \mathsf{proj}(h_I, 1, s), \ldots, \mathsf{proj}(h_I, n{-}1, s))$ where c is the unique constructor.

Two additional completeness rules handle cases where ι-reduction is stuck because the major premise does not reduce to a constructor:

- **K-like reduction**: if I is a Prop type with exactly one constructor and zero fields (e.g., True, or Eq at fixed indices), and the recursor's major premise does not reduce to a constructor application, the unique constructor is synthesized. This is sound because Prop proof irrelevance guarantees exactly one constructor value.
- **Structure expansion**: if I is a single-constructor type with fields (a structure) and the major premise does not reduce to a constructor application, it is expanded to

$$c(\bar{p},\ \mathsf{proj}(h_I, 0, x),\ \ldots,\ \mathsf{proj}(h_I, n{-}1, x))$$

where x is the major premise, enabling ι-reduction to proceed.

CIC enjoys the property that definitional equality is decidable, which is essential for typechecking to be decidable.

2.5 Definitional Equality Algorithm

The definitional equality check $e_1 \equiv e_2$ is the heart of the typechecker. We outline the algorithm, which follows the Lean 4 kernel [1] with the simplification that there is no opacity to manage.

1. **Syntactic equality.** If e_1 and e_2 are syntactically identical (same AST), return true. This short-circuits the common case.
2. **Weak-head normalization (core).** Reduce both sides using $\mathsf{whnf}_{\mathsf{core}}$, which applies β, ζ, ι, projection, and quotient reductions—but *not* δ. Deferring δ-reduction avoids expensive unfolding when possible.
3. **Syntactic equality after $\mathsf{whnf}_{\mathsf{core}}$.** Compare the reduced terms. If equal, return true.
4. **Proof irrelevance.** If both sides have a type in Prop, return true.
5. **δ-reduction.** Unfold constants on both sides (in HashMath, all constants are transparent) and repeat the comparison. When both sides are constants, a heuristic unfolds the "simpler" side first.
6. **Constant equality.** If both sides are $\mathsf{const}(h_1, \bar{u}_1)$ and $\mathsf{const}(h_2, \bar{u}_2)$ with $h_1 = h_2$, check that $\bar{u}_1$ and $\bar{u}_2$ are pointwise equal as levels.
7. **Application congruence.** If both sides are applications $\mathsf{app}(f_1, a_1)$ and $\mathsf{app}(f_2, a_2)$, recursively check $f_1 \equiv f_2$ and $a_1 \equiv a_2$.
8. **η-expansion.** If one side is $\mathsf{lam}(A, b)$ and the other is not, η-expand the non-lambda side and recurse.
9. **Structural η.** If both sides have a structure type, project all fields and compare pairwise.

If none of these steps succeeds, the terms are not definitionally equal.

2.6 Declaration Kinds

Every entry in the HashMath store is a *declaration*. There are four kinds:

Definition 2.3 (Declaration)**.** A declaration is one of:

- $\mathsf{axiom}(k, \tau)$ — an axiom with k universe parameters and type τ. No definition body; the axiom is trusted.
- $\mathsf{definition}(k, \tau, v)$ — a definition with k universe parameters, type τ, and value v. The typechecker verifies $v : \tau$.
- $\mathsf{inductive}(\mathit{block})$ — an inductive type declaration (see Section 2.7).
- $\mathsf{quotient}(\mathit{kind})$ — one of the four built-in quotient constants (see Section 2.8).

In all cases, $k \in \mathbb{N}$ specifies the number of universe parameters. The type τ and value v (when present) may contain $\mathsf{param}(n)$ level expressions for $0 \leq n < k$.

2.7 Inductive Types

An inductive type declaration specifies a family of mutually defined types and their constructors. The *strict positivity* constraint ensures that recursive occurrences of the type being defined appear only in strictly positive positions in constructor argument types, guaranteeing logical consistency [11].

Definition 2.4 (Inductive block)**.** An inductive block is a record:

$$\mathsf{InductiveBlock} \;=\; \{k \in \mathbb{N},\ p \in \mathbb{N},\ \mathit{types} : \mathsf{List\ InductiveType},\ \mathit{isUnsafe} : \mathsf{Bool}\}$$

where k is the number of universe parameters, p is the number of parameters, *isUnsafe* marks the block as unsafe (bypassing the kernel's soundness checks, used in Lean 4 for implementation-internal

types that are not intended to appear in proofs), and each InductiveType is:

$$\mathsf{InductiveType} \;=\; \{\mathit{type} : \mathsf{Expr},\ \mathit{ctors} : \mathsf{List\ Expr}\}$$

The *type* field gives the type former's type, and *ctors* lists the types of each constructor. Mutual inductive types have multiple entries in *types*.

Example 2.1 (Natural numbers)**.** The natural numbers are defined inductively:

$$\mathsf{InductiveBlock}\left\{\begin{array}{l} k = 0,\ p = 0, \\ \mathit{types} = [\{\ \mathit{type} = \mathsf{Type},\ \mathit{ctors} = [\mathsf{Nat},\ \mathsf{Nat} \to \mathsf{Nat}]\ \}], \\ \mathit{isUnsafe} = \mathsf{false} \end{array}\right\}$$

Here Nat : Type is the type former, and the two constructors have types zero : Nat and succ : Nat → Nat. The elimination principle (recursor) is generated automatically.

Example 2.2 (Propositional equality)**.** The identity type Eq is an inductive family with $k = 1$ universe parameter and $p = 1$ parameter:

$$\mathsf{InductiveBlock}\left\{\begin{array}{l} k = 1,\ p = 1, \\ \mathit{types} = [\{\ \mathit{type} = \Pi(A{:}\mathsf{Type}_u).\, A \to A \to \mathsf{Prop}, \\ \qquad\qquad \mathit{ctors} = [\Pi(A{:}\mathsf{Type}_u).\, \Pi(a{:}A).\, \mathsf{Eq}\ A\ a\ a]\ \}], \\ \mathit{isUnsafe} = \mathsf{false} \end{array}\right\}$$

Recursor shape. For each inductive type, the kernel generates a *recursor* (elimination principle) whose type has the form:

$$\mathsf{rec}_I : \Pi\ \bar{p}.\ \Pi\ (C : I\ \bar{p} \to \mathsf{Sort}(j)).\ \bar{m} \to \Pi\ (x : I\ \bar{p}).\ C\ x$$

where $\bar{p}$ are the type's parameters, C is the *motive* (specifying what is to be proved or computed for each element of I), $\bar{m}$ are the *minor premises* (one per constructor, handling recursive arguments via induction hypotheses), and x is the *major premise* (the value being eliminated). The universe j of the motive is subject to the large elimination restriction below. For indexed families (like Eq), the motive additionally abstracts over the index arguments.

Large elimination restriction. Prop is impredicative (one can quantify over all types and remain in Prop) and enjoys proof irrelevance (all proofs of the same proposition are definitionally equal). Together, these features require restricting when a Prop inductive may eliminate into Type—so-called *large elimination*.

Without the restriction, one could define $\mathsf{Wrap}(A : \mathsf{Sort}(u)) : \mathsf{Prop}$ with a single constructor $\mathsf{mk} : A \to \mathsf{Wrap}\ A$, extract data via large elimination ($\mathsf{unwrap} : \mathsf{Wrap}\ A \to A$), and then derive $\mathsf{true} \equiv \mathsf{false}$ from proof irrelevance ($\mathsf{mk\ true} \equiv \mathsf{mk\ false}$ since both inhabit the same Prop).

The rule is: a Prop inductive *allows* large elimination (motive in $\mathsf{Sort}(j)$ for any j) only when every constructor's non-parameter fields have types in Prop. Otherwise, the recursor's motive is restricted to Prop (i.e., $j = 0$). Examples:

- True (one constructor, zero fields) — large elimination allowed.
- Eq (one constructor, zero non-parameter fields) — large elimination allowed. This is what makes Eq.rec useful for rewriting.
- And (one constructor, two Prop fields) — large elimination allowed.

- Or (two constructors) — large elimination **blocked**. One cannot decide which disjunct holds.
- Exists (constructor field in Type) — large elimination **blocked**. One cannot extract the witness.

When the target sort of the inductive type is polymorphic (e.g., $\mathsf{Sort}(u)$ where u could be instantiated to zero), the restriction is applied conservatively: if any instantiation could place the type in Prop with a Type-level field, large elimination is blocked. In mutual inductive blocks, the restriction is applied block-wide: if any type in the block requires restricted elimination, all recursors in the block are restricted.

2.8 Quotient Types

Lean 4 (and therefore HashMath) includes four built-in constants for quotient types, which cannot be defined as ordinary inductive types:

- Quot — the quotient type former:

$$\mathsf{Quot} : (\alpha : \mathsf{Sort}(u)) \to (\alpha \to \alpha \to \mathsf{Prop}) \to \mathsf{Sort}(u)$$

- $\mathsf{Quot.mk}$ — the quotient constructor:

$$\mathsf{Quot.mk} : \{r : \alpha \to \alpha \to \mathsf{Prop}\} \to \alpha \to \mathsf{Quot}\ \alpha\ r$$

- $\mathsf{Quot.lift}$ — lift a function that respects the equivalence relation:

$$\mathsf{Quot.lift} : (\alpha \to \beta) \to (\forall\ a\ b,\ r\ a\ b \to f\ a = f\ b) \to \mathsf{Quot}\ \alpha\ r \to \beta$$

- $\mathsf{Quot.ind}$ — the induction principle:

$$\mathsf{Quot.ind} : (\forall\ a,\ \varphi\ (\mathsf{Quot.mk}\ r\ a)) \to \forall\ q,\ \varphi\ q$$

The *kind* field in a quotient declaration identifies which of these four constants it is. Their computation rules are given in Section 2.4.

2.9 Universe Polymorphism

Modern implementations of CIC (including Lean 4) support *universe polymorphism*: definitions can be parameterized by universe level variables, instantiated at each use site. This avoids code duplication and is essential for a practical foundation.

The universe hierarchy is:

$$\mathsf{Prop}\ :\ \mathsf{Type}(0)\ :\ \mathsf{Type}(1)\ :\ \mathsf{Type}(2)\ :\ \cdots$$

Universe levels are compared using the normalization procedure: reduce max and imax where possible, then check pointwise equality of the resulting normal forms. Universe polymorphism interacts importantly with content-addressing (Section 7.2).

2.10 Transparency Model

In Lean 4, definitions carry an opacity attribute that controls when the definitional equality checker may unfold them (`@[reducible]`, `@[semireducible]`, or `@[irreducible]`). This is a useful performance optimization, but it introduces an elaboration-level concern into the kernel: two terms may or may not be definitionally equal depending on the transparency setting.

HashMath takes a simpler approach: **all definitions are always fully transparent**. The definitional equality checker may unfold any definition at any time. There is no opacity mechanism.

This simplification is justified by the content-addressing design:

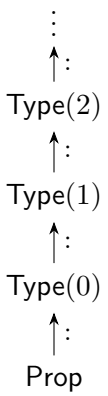


Figure 2: The universe hierarchy. Each sort inhabits the next, forming a cumulative tower. Prop is impredicative: Π-types into Prop remain in Prop.

- Opacity is a *performance* concern, not a *correctness* concern. A term that typechecks with some definitions opaque also typechecks with all definitions transparent (though possibly more slowly).
- Content-addressing requires a canonical notion of definitional equality. If opacity affected which terms are equal, two verifiers with different opacity settings might disagree, breaking the determinism that content-addressing requires.
- The absence of opacity simplifies the kernel and the definitional equality algorithm.

3 Content-Addressed Terms

3.1 The Naming Problem in Detail

Consider the theorem that addition on natural numbers is commutative. In Lean 4's Mathlib, this is `Nat.add_comm`. In Rocq's standard library, it is also `Nat.add_comm`—but the two are not interchangeable because they exist in different systems with different elaboration, different dependencies, and different internal representations. An AI agent might prove the same theorem and call it `theorem_7a3f`. Three correct formalizations of the same mathematical fact, none of which can reference the others.

The problem compounds at scale. Mathlib has undergone multiple renaming campaigns, where thousands of theorems are renamed to conform to evolving conventions. Each renaming breaks downstream projects that depend on the old names. When independent groups formalize overlapping mathematics, reconciliation requires human labor to identify duplicates, choose canonical names, and redirect references.

3.2 Content-Addressing for CIC Terms

We define the hash function $\mathcal{H}$ on CIC terms at the byte level. All hashing uses SHA-256 [16]. Single-byte tags provide domain separation between constructors, natural numbers are encoded as unsigned LEB128, and hashes are fixed 32-byte values. This specification is complete and unambiguous—it uniquely determines the hash of every well-formed term, level, and declaration.

Definition 3.1 (Level hash)**.** The hash of a universe level is:

$$\begin{aligned}
\mathcal{H}(\mathsf{zero}) &= \mathsf{SHA256}(\texttt{0x00}) \\
\mathcal{H}(\mathsf{succ}(l)) &= \mathsf{SHA256}(\texttt{0x01} \parallel \mathcal{H}(l)) \\
\mathcal{H}(\mathsf{max}(l_1, l_2)) &= \mathsf{SHA256}(\texttt{0x02} \parallel \mathcal{H}(l_1) \parallel \mathcal{H}(l_2)) \\
\mathcal{H}(\mathsf{imax}(l_1, l_2)) &= \mathsf{SHA256}(\texttt{0x03} \parallel \mathcal{H}(l_1) \parallel \mathcal{H}(l_2)) \\
\mathcal{H}(\mathsf{param}(n)) &= \mathsf{SHA256}(\texttt{0x04} \parallel \mathsf{leb128}(n))
\end{aligned}$$

Definition 3.2 (Term hash)**.** The hash function $\mathcal{H} : \mathsf{Expr} \to \{0,1\}^{256}$ is:

$$\begin{aligned}
\mathcal{H}(\mathsf{bvar}(n)) &= \mathsf{SHA256}(\texttt{0x10} \parallel \mathsf{leb128}(n)) \\
\mathcal{H}(\mathsf{Sort}(l)) &= \mathsf{SHA256}(\texttt{0x11} \parallel \mathcal{H}(l)) \\
\mathcal{H}(\mathsf{const}(h, [l_0, \ldots, l_{n-1}])) &= \mathsf{SHA256}(\texttt{0x12} \parallel h \parallel \mathsf{leb128}(n) \parallel \mathcal{H}(l_0) \parallel \cdots \parallel \mathcal{H}(l_{n-1})) \\
\mathcal{H}(\mathsf{app}(f, a)) &= \mathsf{SHA256}(\texttt{0x13} \parallel \mathcal{H}(f) \parallel \mathcal{H}(a)) \\
\mathcal{H}(\mathsf{lam}(T, b)) &= \mathsf{SHA256}(\texttt{0x14} \parallel \mathcal{H}(T) \parallel \mathcal{H}(b)) \\
\mathcal{H}(\mathsf{forallE}(T, b)) &= \mathsf{SHA256}(\texttt{0x15} \parallel \mathcal{H}(T) \parallel \mathcal{H}(b)) \\
\mathcal{H}(\mathsf{letE}(T, v, b)) &= \mathsf{SHA256}(\texttt{0x16} \parallel \mathcal{H}(T) \parallel \mathcal{H}(v) \parallel \mathcal{H}(b)) \\
\mathcal{H}(\mathsf{proj}(h, i, e)) &= \mathsf{SHA256}(\texttt{0x17} \parallel h \parallel \mathsf{leb128}(i) \parallel \mathcal{H}(e)) \\
\mathcal{H}(\mathsf{iref}(k, [l_0, \ldots, l_{n-1}])) &= \mathsf{SHA256}(\texttt{0x18} \parallel \mathsf{leb128}(k) \parallel \mathsf{leb128}(n) \parallel \mathcal{H}(l_0) \parallel \cdots \parallel \mathcal{H}(l_{n-1}))
\end{aligned}$$

where h in const and proj is the 32-byte hash of the referenced declaration, and leb128 denotes unsigned LEB128 encoding. Binder names and binder info are excluded: only the type-theoretic content is hashed.

Definition 3.3 (Declaration hash)**.** The hash of a declaration is:

$$\begin{aligned}
\mathcal{H}(\mathsf{axiom}(n, T)) &= \mathsf{SHA256}(\texttt{0x20} \parallel \mathsf{leb128}(n) \parallel \mathcal{H}(T)) \\
\mathcal{H}(\mathsf{definition}(n, T, v)) &= \mathsf{SHA256}(\texttt{0x21} \parallel \mathsf{leb128}(n) \parallel \mathcal{H}(T) \parallel \mathcal{H}(v)) \\
\mathcal{H}(\mathsf{inductive}(\mathit{block})) &= \mathsf{SHA256}(\texttt{0x22} \parallel \mathsf{encode}(\mathit{block})) \\
\mathcal{H}(\mathsf{quotient}(k)) &= \mathsf{SHA256}(\texttt{0x23} \parallel k_{\mathsf{byte}})
\end{aligned}$$

where n is the number of universe parameters and the inductive block encoding follows the same conventions (LEB128 lengths, recursive sub-hashing).

Definition 3.4 (Derived entity hash)**.** Each inductive block generates derived entities—individual types, constructors, and recursors—whose hashes are derived from the block hash:

$$\begin{aligned}
\mathcal{H}(\mathsf{indType}(h_B, i)) &= \mathsf{SHA256}(\texttt{0x30} \parallel h_B \parallel \mathsf{leb128}(i)) \\
\mathcal{H}(\mathsf{indCtor}(h_B, i, j)) &= \mathsf{SHA256}(\texttt{0x31} \parallel h_B \parallel \mathsf{leb128}(i) \parallel \mathsf{leb128}(j)) \\
\mathcal{H}(\mathsf{indRec}(h_B, i)) &= \mathsf{SHA256}(\texttt{0x32} \parallel h_B \parallel \mathsf{leb128}(i))
\end{aligned}$$

where h_B is the block hash, i is the type index within the block, and j is the constructor index within the type.

Remark 3.1. The tag bytes are chosen so that levels (0x00–0x04), expressions (0x10–0x18), the storage-layer hash reference (0x19), declarations (0x20–0x23), and derived entities (0x30–0x32) occupy disjoint ranges. This ensures that no term can collide with a level, declaration, or derived entity at the SHA-256 input level, even before considering structural differences. The 0x19 tag is used only in the stored wire format for href nodes (Section 3.7); it never appears in canonical serialization or hash computation.

Several design decisions are critical:

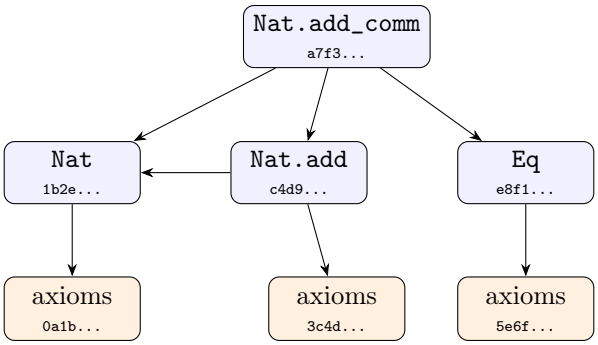


Figure 3: Merkle DAG of `Nat.add_comm`. Each node's hash transitively encodes all dependencies down to the axioms. Human-readable names shown for clarity; the system operates on hashes.

Binder names and binder info are excluded. The terms $\lambda(x{:}\mathsf{Nat}).\, x$ and $\lambda(y{:}\mathsf{Nat}).\, y$ are α-equivalent and receive the same hash, since both have de Bruijn representation $\mathsf{lam}(\mathsf{Nat}, \mathsf{bvar}(0))$. Lean 4's binder info (implicit, explicit, instance) is an elaboration concern and is similarly excluded.

Constants reference by hash. The hash of $\mathsf{const}(h, \bar{u})$ includes the 32-byte hash h of the referenced declaration. This is the key move that eliminates name-dependence: two constants with different names but identical definitions produce the same hash.

Free variables are forbidden. Only *closed* terms—those with no free variables—may be stored in the hashset. Every bound variable must be under a binder, and every constant must resolve to a stored definition.

Example 3.1 (Hashing the identity function on Nat)**.** Consider $\lambda(n{:}\mathsf{Nat}).\, n$, which in de Bruijn notation is $\mathsf{lam}(\mathsf{Nat}, \mathsf{bvar}(0))$. Its hash is:

$$\mathcal{H}(\mathsf{lam}(\mathsf{Nat}, \mathsf{bvar}(0))) = \mathsf{SHA256}(\texttt{0x14} \parallel \mathcal{H}(\mathsf{Nat}) \parallel \mathsf{SHA256}(\texttt{0x10} \parallel \mathsf{leb128}(0)))$$

where $\mathcal{H}(\mathsf{Nat})$ is the hash of the inductive declaration defining Nat.

Example 3.2 (Alpha-equivalence)**.** The terms $\lambda(x{:}\mathsf{Nat}).\, x$ and $\lambda(y{:}\mathsf{Nat}).\, y$ both have de Bruijn representation $\mathsf{lam}(\mathsf{Nat}, \mathsf{bvar}(0))$. Since binder names are excluded from the hash, they produce identical hashes.

3.3 Capturing the Environment

Definition 3.5 (Hash-closed term)**.** A term t is *hash-closed* if every constant reference $\mathsf{const}(h, \bar{u})$ occurring in t can be resolved to a declaration whose hash exists in the hashset $\mathcal{S}$.

Hash-closure means that the hash of any term transitively encodes its entire dependency tree. This creates a **Merkle DAG** of mathematical knowledge: the hash of `Nat.add_comm` encodes, through the hashes of its subterms, the definitions of Nat, $\mathsf{Nat.add}$, Eq, and transitively down to the axioms of the type theory.

3.4 Mutual Recursive Definitions

Mutual recursion presents a challenge: if definition A references B and B references A, then $\mathcal{H}(A)$ depends on $\mathcal{H}(B)$ and vice versa.

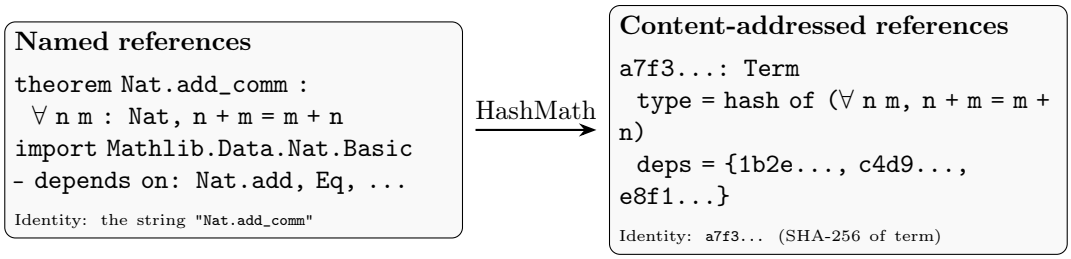


Figure 4: Named references (left) identify terms by human-chosen strings within a specific library. Content-addressed references (right) identify terms by their cryptographic hash, independent of any naming convention.

Definition 3.6 (Mutual block hash)**.** A mutual recursive block $M = (d_1, \ldots, d_k)$ is an ordered list of definitions that may reference each other. The block hash is:

$$\mathcal{H}(M) = \mathsf{SHA256}(\texttt{"mutual"} \parallel \mathcal{H}_{\mathsf{raw}}(d_1) \parallel \cdots \parallel \mathcal{H}_{\mathsf{raw}}(d_k))$$

where $\mathcal{H}_{\mathsf{raw}}(d_i)$ hashes definition d_i with intra-block references represented as block-relative indices rather than hashes. Each individual definition's identity is then the pair $(\mathcal{H}(M), i)$ where i is its index within the block.

This is analogous to how Lean 4 handles `mutual ... end` blocks. The approach preserves the acyclic Merkle DAG property at the block level: blocks reference other blocks (or individual definitions), and no cyclic block dependencies are permitted.

3.5 The Global Hashset

Definition 3.7 (Hashset)**.** The global hashset is a set of verified triples:

$$\mathcal{S} = \{(h, t, T) \mid h = \mathcal{H}(t),\ T = \mathsf{typeof}(t),\ \mathsf{well_typed}(t)\}$$

together with a *type index*:

$$\mathcal{I} : \{0,1\}^{256} \to \mathcal{P}(\{0,1\}^{256}), \qquad \mathcal{I}(h_T) = \{h \mid (h, t, T) \in \mathcal{S},\ \mathcal{H}(T) = h_T\}$$

mapping each type hash to the set of hashes of its inhabitants.

The hashset is *append-only*: once a verified triple is added, it is never removed. Since well-typedness is verified before insertion, every entry in the hashset is correct by construction.

3.6 Comparison with Unison

The Unison programming language pioneered content-addressing for general-purpose code, hashing abstract syntax trees with 512-bit SHA3 [14]. HashMath uses SHA-256 [16] rather than SHA3. HashMath adapts this approach for the specific demands of formal mathematics:

- **Type-to-inhabitants index**: Unison does not maintain a global map from types to their inhabitants, because general-purpose code does not have the same "search by type" need.
- **Formal verification**: every term in HashMath is type-checked before insertion; Unison stores arbitrary well-scoped code without a comparable verification step.
- **Universe polymorphism**: CIC's universe polymorphism requires careful treatment in the hash function (Section 7.2).

- **Inductive types**: CIC's inductive types, with strict positivity and elimination principles, have no analog in Unison's type system.

Unison's analysis of collision probability applies equally to HashMath: with 2^{128} security against collision attacks (the birthday bound for a 256-bit hash), the probability of a collision in a library of 2^{40} terms (one trillion) is negligible ($\approx 2^{-176}$).

3.7 Subterm-Level Hash-Consing

The hashing scheme described above assigns a content hash to each *declaration*. But CIC proof terms are deeply nested trees that often share subterms—the type $\mathsf{Nat} \to \mathsf{Nat}$ may appear thousands of times across a library. At the declaration level, these shared subterms are duplicated in every serialized declaration that contains them.

HashMath extends the Merkle DAG from declarations down to *subterms*. When a declaration is prepared for storage or transmission, a *shatter* operation replaces large subterms with href nodes carrying the subterm's content hash. Each replaced subterm is serialized independently and stored as a separate entry in the DHT, keyed by its hash.

Definition 3.8 (Shatter and reassemble)**.** Let $\theta \in \mathbb{N}$ be a size threshold (the reference implementation uses $\theta = 33$, the cost of one href node: 1 tag byte + 32 hash bytes). The *shatter* operation $\mathsf{shatter}_\theta : \mathsf{Expr} \to \mathsf{Expr}_{\mathsf{Hash}} \times \mathsf{SubtermStore}$ traverses a closed expression bottom-up. For each subterm e whose canonical serialization exceeds θ bytes:

1. Compute $h = \mathcal{H}(e)$ and the canonical byte serialization $b = \mathsf{serialize}(e)$.
2. Store the mapping $h \mapsto b$ in the $\mathsf{SubtermStore}$.
3. Replace e in the parent with $\mathsf{href}(h)$.

The inverse operation $\mathsf{reassemble} : \mathsf{Expr}_{\mathsf{Hash}} \times \mathsf{SubtermStore} \to \mathsf{Expr}$ resolves every $\mathsf{href}(h)$ by deserializing the corresponding entry from the store.

Phantom type safety. The expression type is parameterized as $\mathsf{Expr}(\alpha)$. Kernel operations (type inference, definitional equality, reduction) work with $\mathsf{Expr}(\bot)$, where $\bot$ is an empty type. Since $\mathsf{href} : \alpha \to \mathsf{Expr}(\alpha)$ requires an inhabitant of α, and $\bot$ has none, the href constructor is *statically uninhabitable* in kernel code—no match arm, no runtime check, no possibility of a hash reference reaching the type checker. The storage layer works with $\mathsf{Expr}(\mathsf{Hash})$, where Hash is the 32-byte digest type. An embedding $\mathsf{Expr}(\bot) \hookrightarrow \mathsf{Expr}(\alpha)$ is provided for safe upcast.

Hash preservation. Shattering does not change a term's content hash:

$$\mathcal{H}(e) \;=\; \mathcal{H}\big(\mathsf{reassemble}(\mathsf{shatter}_\theta(e))\big)$$

because $\mathsf{reassemble}$ recovers the original term exactly and canonical serialization is deterministic. The stored subterm entries use the *canonical* serialization (no href nodes), so each entry is self-contained.

Deduplication. Subterm entries are keyed by content hash. If two declarations share a subterm, both produce the same hash for that subterm, and the DHT stores it once. At scale, this provides global subterm compression: the $\mathsf{Nat} \to \mathsf{Nat}$ type, the recursor for List, or a standard proof pattern that recurs across thousands of declarations is stored *exactly once* in the entire network.

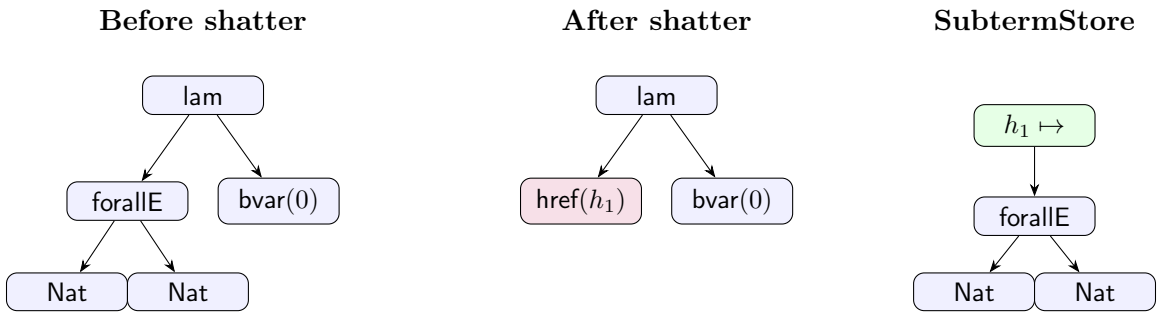


Figure 5: Subterm-level hash-consing. The identity function $\lambda(n{:}\mathsf{Nat}{\to}\mathsf{Nat}).\, n$ is shattered: the domain $\mathsf{Nat} \to \mathsf{Nat}$ (which exceeds the 33-byte threshold) is replaced by a hash reference $\mathsf{href}(h_1)$ and stored separately. If another declaration shares the same subterm, it references the same h_1 entry—stored once globally.

Wire format. Stored declarations use a variant serialization in which href nodes are encoded with tag `0x19` followed by 32 raw hash bytes. All other constructors use the same tags as canonical serialization. The format is backward-compatible: declarations without href nodes are valid in both the canonical and stored formats, so older nodes that predate subterm hash-consing can still be read without modification.

4 Verification and Trust

4.1 The Verification Protocol

When a term t is submitted to the hashset, the following protocol executes:

1. **Parse**: deserialize t from the binary format into a CIC term.
2. **Resolve**: verify that every constant reference in t has a corresponding entry in $\mathcal{S}$.
3. **Typecheck**: type-check t in the CIC, using the referenced definitions as the environment. If t is ill-typed, reject.
4. **Hash**: compute $h = \mathcal{H}(t)$ and $h_T = \mathcal{H}(\mathsf{typeof}(t))$.
5. **Store**: add $(h, t, \mathsf{typeof}(t))$ to $\mathcal{S}$ and h to $\mathcal{I}(h_T)$.

If any step fails, the term is rejected and no state changes. The protocol is *deterministic*: given the same hashset state and the same term, every compliant node produces the same result.

4.2 Trust Properties

The verification protocol establishes several properties:

Theorem 4.1 (Correctness)**.** *Every term $(h, t, T) \in \mathcal{S}$ satisfies: t is well-typed in CIC with type T, and all dependencies of t are also in $\mathcal{S}$.*

This follows directly from the protocol: step 3 verifies well-typedness, step 2 verifies that all dependencies exist, and the append-only nature of the hashset ensures that dependencies are never invalidated.

Additional trust properties:

- **Integrity**: SHA-256 collision resistance ensures that distinct terms receive distinct hashes, with overwhelming probability.
- **Immutability**: a hash pins exact content; the term at hash h can never change.

- **Composability**: any well-typed term can reference any existing term in $\mathcal{S}$ by hash, regardless of who contributed either term.
- **No trust in contributors**: verification is mechanical. A term submitted by an untrusted AI agent is verified identically to one submitted by a Fields medalist—and both are equally trustworthy once verified.

4.3 The Small Trusted Computing Base

The system's correctness depends only on:

1. the CIC type checker being correct,
2. the SHA-256 implementation being correct, and
3. the serialization format faithfully representing terms.

The CIC type checker is remarkably small. Diehl [12] demonstrates that dependent type checking can be implemented in approximately 200 lines of Python, and his "Typechecker Zoo" [13] catalogs minimal implementations across languages. This is the *de Bruijn criterion* [17]: a small, auditable kernel through which all trust flows.

Crucially, this kernel can itself be formally verified. Sozeau et al. [19] demonstrated in the MetaCoq project that the CIC type checker can be formalized *within CIC itself* and proved correct—"Coq Coq Correct!" Their verified type checker, proved sound with respect to a formal specification of CIC, found and fixed a real incompleteness bug in Coq 8.14. This shifts the trust from a *trusted code base* (TCB) to a *trusted theory base* (TTB): one need not trust the implementation, only the mathematical specification of the type theory.

HashMath could adopt MetaCoq's verified checker, or a similar verified kernel, to minimize the trusted computing base to the bare mathematical theory.

Remark 4.1. The de Bruijn criterion was articulated by Pollack [18]: "How to believe a machine-checked proof." The answer is to make the proof-checking kernel so small that it can be inspected and verified independently of the proof-generating machinery.

5 The Knowledge Base Architecture

5.1 Data Model

The knowledge base consists of three components:

$$\begin{aligned}
\mathcal{S} &= \{(h, t, T) \mid h = \mathcal{H}(t),\ T = \mathsf{typeof}(t),\ \mathsf{well_typed}(t)\} \\
\mathcal{I} &= \{h_T \mapsto \{h \mid (h, t, T) \in \mathcal{S},\ \mathcal{H}(T) = h_T\}\} \\
\mathcal{N} &= \{\mathit{name} \mapsto h\} \quad \text{(metadata, not core)}
\end{aligned}$$

The hashset and type index are the core infrastructure; name maps are an optional layer maintained by communities or individuals.

5.2 Operations

The knowledge base supports five primary operations:

- $\mathsf{submit}(t)$: verify t and add to $\mathcal{S}$; return $\mathcal{H}(t)$.
- $\mathsf{lookup}(h)$: retrieve the term and type at hash h.
- $\mathsf{search_by_type}(h_T)$: return $\mathcal{I}(h_T)$—all inhabitants of type T.
- $\mathsf{dependencies}(h)$: compute the transitive dependency set by walking the Merkle DAG.
- $\mathsf{name}(h, n)$: associate human-readable name n with hash h (metadata operation).

5.3 Storage and Distribution

The hashset is designed for decentralized storage and distribution:

- **Replication**: any node can maintain a full or partial copy of $\mathcal{S}$, similar to Git or IPFS [23].
- **Independent verification**: any node can verify any term independently, requiring only the term's transitive dependencies.
- **No central authority**: there is no privileged node; all nodes are equal.
- **Incremental syncing**: a node need only fetch hashes it does not already have. Since hashes are deterministic, two nodes with the same hash have identical content.

Terms are serialized in a compact binary format for storage and transmission. The serialization must be canonical (deterministic) to ensure that hash computation is consistent across implementations. Subterm-level hash-consing (Section 3.7) extends this model to individual subterms, enabling fine-grained deduplication across the entire knowledge base.

5.4 The Type-to-Inhabitants Index

The type index $\mathcal{I}$ enables a powerful new mode of mathematical search. Given a type—which is itself a proposition under the Curry-Howard correspondence—one can ask: "what are all known proofs of this proposition?"

Example 5.1. To find all proofs that $\forall(n : \mathsf{Nat}),\ n + 0 = n$, one computes the hash of the type $\Pi(n{:}\mathsf{Nat}).\,\mathsf{Eq}\ \mathsf{Nat}\ (\mathsf{Nat.add}\ n\ \mathsf{Nat.zero})\ n$ and queries $\mathcal{I}$ for that hash. The result includes every proof of this fact in the entire knowledge base, regardless of who contributed it or what they named it.

This is analogous to Hoogle for Haskell or Moogle for Lean, but operating globally across all contributions. For AI agents, the type index transforms proof search from a name-recall problem into a type-matching problem—a far more natural interface for machine reasoning.

5.5 Interoperability Layer

HashMath is designed to interoperate with existing proof assistants through export pipelines:

Lean 4 pipeline. Lean 4's kernel expressions are CIC terms. An export tool translates Lean `Expr` values to HashMath's canonical representation, computes hashes, and submits to the hashset. This pipeline can import the entirety of Mathlib.

Rocq pipeline. Rocq's Gallina is also CIC, but the kernels diverge in important ways (see Section 7.10): Rocq uses `match` instead of recursors, lacks primitive quotient types, has different η rules, and includes the Set universe. A Rocq export tool must translate these kernel differences into HashMath CIC's canonical form. Terms that rely on Rocq-specific kernel features may not have direct HashMath equivalents; in such cases, the export pipeline must re-elaborate or reject the term. Cross-system sharing of proof hashes requires either a translation layer or convergence on shared elaboration patterns—both are feasible but nontrivial engineering challenges.

Contrast with Logipedia/Dedukti. The Logipedia project [20] and the Dedukti logical framework [21] take a different approach: they translate proofs between different type theories through a common framework (the $\lambda\Pi$-calculus modulo theory). The EuroProofNet COST action [22] (2022–2025, 600+ researchers, 47 countries) has advanced this vision. HashMath bypasses the translation problem by working at the CIC kernel level where Lean and Rocq agree on the core theory, though

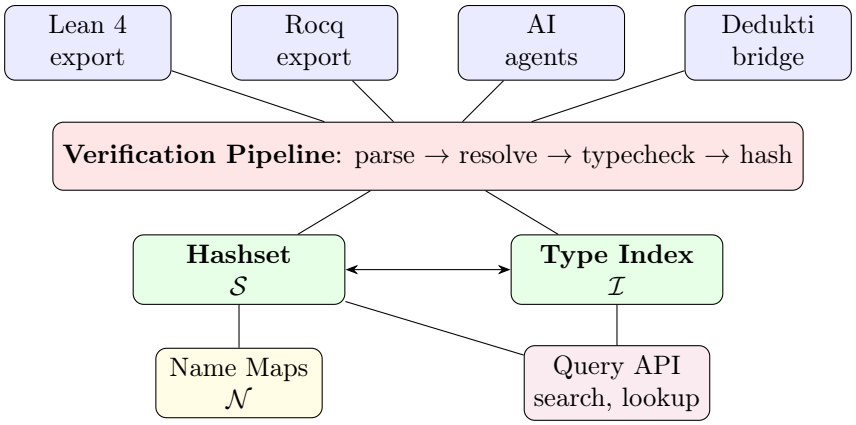


Figure 6: HashMath architecture. Terms from multiple sources pass through the verification pipeline before being added to the hashset and type index. Name maps are optional metadata. The query API supports lookup by hash and search by type.

they diverge on several kernel-level details (Section 7.10). The approaches are complementary: Dedukti could translate non-CIC proofs (e.g., from Isabelle/HOL) into CIC for HashMath storage.

6 AI and Human Collaboration

6.1 The Coordination Problem Today

Tao's formalization of the Polynomial Freiman-Ruzsa conjecture in Lean involved over 20 contributors, with manual coordination through GitHub issues, Zulip chat, and personal communication [5]. Mathlib contributions require human review, conformance to naming conventions, and compliance with style linters. These processes ensure quality but limit throughput.

AI systems like AlphaProof [3] and DeepSeek-Prover-V2 [4] can produce correct proofs at machine speed, but cannot contribute them to shared libraries without human intermediation: someone must name the theorem, choose the right namespace, write documentation, and submit a pull request. The Polymath project [6] demonstrated the vision of permissionless mathematical collaboration—40+ contributors proving the density Hales-Jewett theorem through blog comments—but the coordination was entirely ad hoc.

6.2 HashMath Eliminates Coordination

In HashMath, the contribution workflow is:

1. Agent (AI or human) constructs a well-typed CIC term t.
2. Agent submits t to the hashset.
3. The verification pipeline checks t and computes $\mathcal{H}(t)$.
4. t is available globally, immediately, by hash.

No pull request. No naming debate. No review queue. Another agent that needs a lemma of type T queries $\mathcal{I}(\mathcal{H}(T))$ and finds t by its type, not by any name.

When multiple agents independently prove the same theorem with the same proof term, they produce the same hash—the duplicate is automatically detected. When they produce different proofs of the same theorem, both are stored and discoverable through the type index. Neither outcome wastes work.

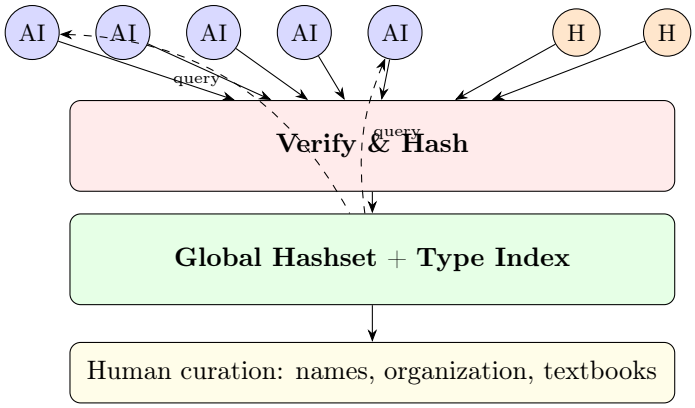


Figure 7: AI agents and humans contribute proof terms to the verification pipeline. Verified terms enter the global hashset. Agents discover existing results through type-directed queries. Humans add curation—names, organization, exposition—as an optional layer.

6.3 The Naming Layer as Social Convention

Names remain useful—for human communication, documentation, textbooks, and teaching. In HashMath, names are metadata: a mapping from strings to hashes, maintained by communities.

- The French school: "théorème de Bézout" $\mapsto$ `a7f3...`
- The English school: "Bezout's theorem" $\mapsto$ `a7f3...`
- An AI agent: `h_7f3a2b` $\mapsto$ `a7f3...`

All three reference the same hash. Name collisions are impossible because names are not identity—they are aliases. Different communities can maintain their own naming conventions without conflict.

6.4 Scaling Mathematics

Tao's vision of "big theorems proved by 20 people and multiple AIs" requires infrastructure that does not exist today. HashMath provides it.

Imagine 1000 AI agents exploring different corners of mathematics simultaneously. Each agent submits verified terms to the hashset. Other agents discover these terms by type signature and build on them. Human mathematicians curate, name, and organize the accumulating knowledge—but the raw content grows permissionlessly, limited only by the rate at which correct proofs can be produced.

7 Challenges and Limitations

We address the principal challenges.

7.1 Definitional Equality and Hash Diversity

Two CIC terms that are *definitionally equal* but *syntactically different* receive different hashes. For example, $(\lambda(x{:}\mathsf{Nat}).\, x)\ \mathsf{zero}$ and zero are definitionally equal but hash differently. This is by design: the hash captures the exact proof term, not just the proposition it proves. The type index is the correct mechanism for finding "equivalent" results—all proofs of a given type are discoverable regardless of their syntactic form.

One could normalize terms before hashing to reduce diversity, but normalization is computationally expensive (exponential in the worst case) and would change the semantics: distinct proof strategies would become indistinguishable. We judge that hashing terms as-is, combined with the type index for discovery, is the right tradeoff.

7.2 Universe Polymorphism

Universe-polymorphic definitions are parameterized by universe level variables. The hash must capture universe parameters correctly: a definition instantiated at $\mathsf{Type}(0)$ and the same definition instantiated at $\mathsf{Type}(1)$ are different terms with different hashes. The hash function handles this through $\mathcal{H}_{\mathsf{level}}$ and $\mathcal{H}_{\mathsf{levels}}$, which hash universe level expressions (variables, successors, max) structurally. Care is needed to ensure that universe constraints are consistently captured.

7.3 Axiom Compatibility

Different mathematical traditions employ different axioms: classical logic vs. constructive logic, the axiom of choice, propositional extensionality, function extensionality, quotient types, and so on. In HashMath, axiom dependencies are carried transitively through the Merkle DAG. Every axiom used in a proof is visible by inspecting the transitive dependencies of its hash.

This enables powerful filtering: "show me all proofs of X that do not use the axiom of choice" is a computable query—walk the Merkle DAG and check whether the axiom of choice's hash appears in the dependency set. Different communities can maintain different axiom policies without preventing contribution to the shared hashset.

7.4 Hash Stability Across Format Versions

If the hashing scheme is ever modified—for example, to add support for a new term constructor—all hashes change. The Unison project experienced this challenge in migrating between hash format versions [14].

We mitigate this through several mechanisms:

- **Version prefix**: a future version of the format could prepend a version byte to every hash input, so that terms self-identify their hashing era. (The current tag bytes in Section 3.2 are constructor tags, not version identifiers.)
- **Bidirectional mappings**: during transitions, maintain a mapping between old and new hashes.
- **Stability as a goal**: the hash format should be designed carefully and changed rarely. Getting it right early is a priority.

7.5 Mutual Recursion

Block-level hashing (Section 3.4) ensures acyclicity but introduces granularity concerns. If a mutual block is large, any modification to one definition changes the block hash and thus the identity of all definitions in the block. In practice, mutual blocks in CIC tend to be small (2–5 definitions), making this manageable. Refactoring a mutual block into a non-mutual formulation (when possible) is a content-level change and correctly results in a different hash.

7.6 Performance

Hashing deep dependency trees requires traversing large Merkle DAGs. Several standard techniques address this:

- **Caching**: once a term's hash is computed, it is stored and never recomputed.
- **Subterm hash-consing**: shared subterms are stored once in the DHT and referenced by hash (Section 3.7), reducing both storage and network transfer.
- **Incremental verification**: when verifying a new term, only the new term and its immediate definition need type-checking; dependencies have already been verified.

7.7 Social Adoption

A content-addressed knowledge base is only useful if it contains content. This is a chicken-and-egg problem with a clear bootstrap strategy:

- **Import Mathlib**: Lean 4's Mathlib contains 210,000+ theorems and is CIC-based. Importing it provides a substantial initial corpus.
- **AI agent tooling**: provide APIs and tooling that make it easy for AI proving systems to submit and query the hashset.
- **Compatibility**: HashMath is complementary to existing systems, not a replacement. Lean and Rocq users can continue using their preferred tools; HashMath is the storage and discovery layer.

7.8 Eta-Equivalence and Proof Irrelevance

HashMath CIC includes function η-equivalence ($f \equiv \lambda x.\, f\ x$), structural η for structure-like inductives, and proof irrelevance for Prop. These features are necessary for compatibility with Lean 4's kernel, but they increase the complexity of the definitional equality checker and the potential for hash diversity.

Two terms that are η-equivalent receive different hashes—the hash captures the exact syntactic structure, not the equivalence class. However, both terms typecheck against the same type and are therefore both discoverable through the type index. This is an intentional design choice: normalizing before hashing would require choosing a canonical form, which is computationally expensive and may not be unique for all equivalences.

7.9 Elaboration Non-Determinism

Different proof assistants (and different versions of the same assistant) may elaborate the same mathematical statement into different kernel terms. Lean 4's tactic mode, for instance, often produces terms with more let-bindings, casts, and auxiliary definitions than a hand-written proof term. Since HashMath hashes the *elaborated* term, the same mathematical theorem may receive different hashes when elaborated by different tools.

This is inherent to any content-addressing scheme that operates below the elaboration level. The type index mitigates the issue: all proofs of a given type are discoverable regardless of their elaboration strategy. Over time, as community norms develop around preferred elaboration patterns, convergence is expected for core mathematical content.

7.10 Lean/Rocq Kernel Divergences

While both Lean 4 and Rocq implement CIC, their kernels differ in several important ways:

- **Recursors vs. match**: Lean 4 uses recursors as the primitive elimination form; Rocq uses `match` expressions. These are interdefinable but produce different kernel terms.
- **Quotient types**: Lean includes quotient types as a primitive; Rocq constructs them using the axiom of unique choice or via setoid reasoning.
- **Eta rules**: Lean includes function η and structural η for structures; Rocq's definitional equality rules differ slightly.
- **Proof irrelevance**: Lean has proof irrelevance for Prop at the kernel level; in Rocq, this holds only for propositional content in the SProp universe.
- **Literals**: Lean has native literal expressions for natural numbers and strings; HashMath CIC eliminates these in favor of $\mathsf{Nat.zero}$/$\mathsf{Nat.succ}$ encoding.
- **No Set sort**: Lean does not have Rocq's Set universe.

These divergences mean that a theorem elaborated in Lean and the "same" theorem elaborated in Rocq will generally receive different hashes. Cross-system sharing requires either a translation layer or a shared elaboration pipeline—both are feasible but represent engineering work beyond the scope of the initial system.

7.11 Spam, DoS, and Proof Term Bloat

An open, permissionless system is vulnerable to spam: adversaries can submit arbitrarily many well-typed but useless terms (e.g., $\mathsf{Nat.succ}^{10^6}(\mathsf{zero})$). This is a *policy* concern, not a specification concern. Possible mitigations include:

- **Size bounds**: reject terms above a configurable size threshold.
- **Rate limiting**: bound the submission rate per identity or IP address.
- **Proof-of-work**: require a computational puzzle alongside each submission.
- **Reputation**: weight search results by community endorsement.

None of these mechanisms are part of the CIC specification; they belong to the policy layer of any deployment.

Relatedly, tactic-elaborated proof terms can be very large—a simple `omega` call may expand to thousands of kernel steps. The subterm-level hash-consing mechanism (Section 3.7) directly addresses this: when a declaration is shattered for storage, shared subterms across all declarations in the network are stored exactly once. A type like $\mathsf{Nat} \to \mathsf{Nat}$ or a recursor application pattern that recurs in thousands of proofs occupies a single DHT entry, referenced by hash everywhere else. This transforms the storage cost from $O(n \cdot s)$ (where n is the number of declarations and s is the average subterm size) to $O(n + s')$ (where s' is the number of *unique* subterms), providing significant compression for large libraries of tactic-elaborated proofs.

7.12 Axiom Identity Across Systems

Lean's `propext` (propositional extensionality) and Rocq's propositional extensionality axiom express the same mathematical principle but, as different kernel terms, receive different hashes. This means that proofs using Lean's `propext` and proofs using Rocq's equivalent cannot share a dependency tree.

Standardizing axiom declarations across systems is future work. One approach is to define a canonical HashMath CIC declaration for each standard axiom and require export pipelines to map system-specific axioms to these canonical forms.

7.13 Type Index Precision

The type index maps type hashes to inhabitant hashes, using *syntactic* type hashes. This means that two types that are definitionally equal but syntactically different (e.g., $\mathsf{Nat} \to \mathsf{Nat}$ and $\Pi(n{:}\mathsf{Nat}).\,\mathsf{Nat}$) produce different index entries. A query for one will not find inhabitants typed at the other.

Normalizing types before indexing could improve precision but introduces computational cost and the question of which normal form to use. In practice, most mathematical types are stated in a relatively canonical form, and the type index is effective despite this limitation.

8 Related Work

8.1 Proof Assistants

Lean 4 [1], Rocq (formerly Coq) [8], Agda, and Isabelle/HOL are the dominant proof assistants. All use named references within curated libraries. HashMath is complementary: a content-addressed storage layer that any CIC-based system can read from and write to, without modifying the system itself.

8.2 Unison

Unison [14] is the closest precedent: it content-addresses general-purpose functional programs using cryptographic hashes of abstract syntax trees. Section 3.6 discusses the detailed comparison; the key differences are that HashMath adds a type-to-inhabitants index, requires formal verification before storage, and handles CIC-specific features (universe polymorphism, inductive types, strict positivity).

8.3 Logipedia and Dedukti

The Logipedia project [20] and Dedukti [21] translate proofs between different type theories through the $\lambda\Pi$-calculus modulo theory. HashMath takes a complementary approach, as discussed in Section 5.5.

8.4 MetaCoq

MetaCoq [19] formalizes the Calculus of Inductive Constructions within Coq itself, including a verified type checker proved sound with respect to a formal specification. This demonstrates that the CIC kernel—the core of HashMath's trusted computing base—can be formally verified. HashMath could adopt MetaCoq's verified checker to minimize trust assumptions.

8.5 IPFS and Content-Addressed Storage

IPFS [23] provides content-addressed storage for arbitrary data. HashMath's storage model is similar but domain-specific: it verifies terms before storing them (IPFS stores any data) and maintains a type index that is meaningful only for CIC terms. HashMath could use IPFS as an underlying storage layer.

8.6 Formal Mathematics Libraries

Mathlib [2], the Archive of Formal Proofs, and Rocq's standard library are curated, centralized, and name-dependent. They represent enormous investments of human effort. HashMath is uncurated, decentralized, and name-independent—but can import these libraries wholesale as its initial corpus.

8.7 AI for Mathematics

AlphaProof [3], DeepSeek-Prover-V2 [4], LeanDojo, and COPRA produce proofs within existing proof assistants. HashMath provides a system-agnostic destination for AI-generated proofs, and the type index provides a natural interface for AI proof search: search by type signature rather than by name.

9 Reference Implementation: Distribution Prototype

The reference implementation includes a working peer-to-peer distribution layer built as a Rust sidecar process (`hm-net`) communicating with the Lean `hm` CLI. This section describes the architecture and protocol of this prototype, which realizes the distribution model described in Section 5.3.

9.1 Architecture

The system follows a two-process architecture with *separated discovery and transfer*, analogous to BitTorrent's Mainline DHT and IPFS's Bitswap:

- **Lean process (`hm`)**: handles parsing, elaboration, type checking, hashing, and serialization. All trust-critical operations remain in Lean.
- **Rust sidecar (`hm-net`)**: runs a libp2p [26] node combining Kademlia DHT [27] for *provider discovery* (which peer has hash h?) with a dedicated request-response protocol (`/hashmath/transfer/1.0.0`) for *direct content transfer* between peers. The sidecar is entirely untrusted—it transports opaque bytes whose integrity is verified by the Lean process upon retrieval.

The key insight is that Kademlia is designed as a lookup-heavy, write-light system. Storing full records (up to 1 MB) in the DHT requires replicating to ∼20 peers per write, limiting throughput to ∼5–20 completions/sec. By storing only lightweight *provider records* (∼40 bytes each) in the DHT and serving content directly, the system achieves much higher throughput: publishing hundreds of declarations completes in a single batch IPC call, with provider announcements propagating asynchronously at a rate-limited 20/sec in the background.

The Lean process spawns `hm-net` as a child process and communicates over stdin/stdout using a binary IPC protocol. The sidecar's libp2p stack includes Kademlia for provider discovery, the Identify protocol for peer metadata exchange, AutoNAT for NAT status detection, and a Relay client for NAT traversal.

9.2 IPC Protocol

Communication uses length-prefixed binary frames:

$$\underbrace{\text{4-byte BE length}}_{L} \parallel \underbrace{\text{tag byte}}_{1} \parallel \underbrace{\text{payload}}_{L-1}$$

Six request types flow from Lean to Rust (tags `0x50`–`0x55`: Ping, Publish, Fetch, GetPeers, Shutdown, PublishBatch) and seven response types flow back (tags `0x60`–`0x66`: Pong, Published,

Found, NotFound, PeerList, Error, BatchPublished). Hash values are transmitted as raw 32-byte SHA-256 digests; variable-length fields use LEB128 length prefixes.

9.3 Publish and Fetch

Publishing. When the user runs `hm publish file.hm`, the Lean process type-checks all declarations, then *shatters* each declaration (Section 3.7): large subterms are replaced by href nodes and stored as separate entries. All fragments (subterms and top-level declarations) are sent to the sidecar in a single `PublishBatch` IPC call. The sidecar stores each record locally (persisted to disk as a file keyed by the hex-encoded hash) and returns immediately. In the background, the sidecar announces provider records to the DHT via Kademlia `START_PROVIDING` at a rate-limited 20 announcements per second, so the network gradually learns which peers hold which content.

Fetching. When the user runs `hm fetch <hash>`, the sidecar first checks its local file store. If the content is not local, it queries the DHT via `GET_PROVIDERS` to discover which peers hold the requested hash, then transfers the content directly from a provider using the `/hashmath/transfer/1.0.0` request-response protocol. A fallback `GET_RECORD` path provides backward compatibility with older nodes that stored full records in the DHT. The Lean process inspects the returned bytes for href nodes (tag `0x19`), fetches each referenced subterm, builds a SubtermStore, and *reassembles* the full declaration. It then recomputes the SHA-256 hash to verify integrity, recursively fetches all declaration-level dependencies (referenced hashes embedded in the term), type-checks each declaration, and adds it to the environment. The process is backward-compatible: declarations without href nodes (published before subterm hash-consing was added) are handled identically to canonical-format records. This ensures that *no ill-typed term can enter the environment through the network*—the sidecar is a transport layer only, and the Lean process independently verifies every declaration.

9.4 Persistent Storage

The sidecar maintains a file-backed record store at a platform-specific data directory. Each Kademlia record is stored as a file named by the hex-encoded key under a `records/` subdirectory. An Ed25519 keypair is persisted at `keypair`, giving the node a stable PeerId across restarts.

9.5 Manifest-Based Bulk Sync

For synchronizing entire libraries, HashMath supports *manifest files* (`.hmm`): plain-text files listing one `name hexhash` pair per line. The `hm manifest -generate` command produces a manifest from source files, and `hm fetch -manifest lib.hmm` fetches and verifies every entry. This enables efficient bulk replication of curated libraries across nodes.

10 Roadmap

We outline a phased development plan, noting progress to date.

Phase 1: Core Infrastructure (complete)**.** The CIC type checker, SHA-256 hash function, canonical binary serialization, and content-addressed environment are implemented in Lean 4 with no external dependencies. The implementation includes over 90 automated tests: 34 test groups in

the internal test suite, 24 correct example files, and 32 files that must be rejected—covering type checking, positivity, universe constraints, large elimination, and error rejection.

Phase 2: Distribution Prototype (complete)**.** A working peer-to-peer distribution layer is implemented using a Rust sidecar with libp2p (Section 9). The architecture separates discovery (Kademlia provider records) from content transfer (direct peer-to-peer `/hashmath/transfer/1.0.0` protocol), following the same pattern as BitTorrent and IPFS. Batch IPC and background rate-limited provider announcements enable high-throughput publishing. The CLI supports publishing, fetching with recursive dependency resolution, peer discovery, and manifest-based bulk sync. Persistent file-backed storage, externalized bootstrap configuration, and NAT traversal support are included. Subterm-level hash-consing (Section 3.7) provides global deduplication: declarations are shattered into subterm fragments before publishing, enabling shared subterms across the entire network to be stored exactly once.

Phase 3: Import Pipeline. Build a Lean 4 export pipeline that translates `Expr` to HashMath terms, hashes them, and populates the hashset. Import Mathlib as the initial corpus (210,000+ theorems). Build a Rocq export pipeline for the standard library.

Phase 4: AI Integration. Provide an API for AI agents to submit verified terms and query the type index. Build a type-directed search interface. Integrate with existing AI proving systems (DeepSeek-Prover, LeanDojo, etc.).

Phase 5: Scaling and Ecosystem. Implement connection limits, rate limiting, and spam resistance. Build IDE integration (VS Code, Emacs) for browsing and searching the hashset. Build a web interface for exploration. Support community-maintained name maps. Develop proof visualization tools.

11 Future Work: Advanced Indices

The type-to-inhabitants index (Section 5.4) is the simplest and most fundamental index over the hashset: given a type hash, return all inhabitants. But a content-addressed knowledge base admits a much richer family of indices, each enabling a different mode of mathematical search. We outline several directions.

11.1 Dependency and Reverse-Dependency Index

Every declaration's hash transitively encodes its dependencies through the Merkle DAG. Materializing this as an explicit index yields two maps:

$$\mathsf{deps}(h) = \{h' \mid h' \text{ is a transitive dependency of } h\}$$

$$\mathsf{rdeps}(h) = \{h' \mid h \in \mathsf{deps}(h')\}$$

The dependency map answers "what does this theorem rely on?"—useful for axiom auditing, minimality checking, and understanding proof structure. The reverse-dependency map answers "what builds on this definition?"—essential for impact analysis and for understanding the downstream consequences of a change.

Both maps are computable from the hashset contents and can be maintained incrementally as new terms are added.

11.2 Axiom Filter Index

Section 7.3 noted that axiom dependencies are visible through the Merkle DAG. A dedicated axiom filter index makes this query efficient:

$$\mathsf{axioms}(h) = \{h' \in \mathsf{deps}(h) \mid h' \text{ is an axiom}\}$$

This enables queries like "find all proofs of X that are constructive" (no excluded middle), "find all proofs that avoid the axiom of choice," or "find all proofs that use only the axioms shared by Lean and Rocq." For communities with specific foundational commitments, the axiom filter index provides automatic compliance checking.

11.3 Unification-Based Type Search

The type-to-inhabitants index requires an *exact* type hash match. This is limiting: the query type must be syntactically identical to the stored type, modulo alpha-equivalence. A more powerful search allows the query type to contain metavariables (wildcards) that are instantiated by unification.

For example, a query for $\forall(n : \mathsf{Nat}),\ ?P\ n \to ?P\ (\mathsf{succ}\ n)$ would match any induction step, returning inhabitants whose types unify with the pattern. This is analogous to Hoogle's type search with polymorphism.

Implementing unification-based search over a large corpus requires indexing techniques from the term-indexing literature [25]: discrimination trees, substitution trees, or path indexing. Adapting these to CIC terms with dependent types is a nontrivial but tractable engineering challenge.

11.4 Head-Symbol Index

Many mathematical queries are about a specific concept: "find all lemmas about List" or "find all properties of Ring." A head-symbol index maps the outermost constant of a type to its inhabitants:

$$\mathsf{head}(h_c) = \{h \mid (h, t, T) \in \mathcal{S},\ \mathsf{headConst}(T) = h_c\}$$

where headConst extracts the outermost constant after stripping universal quantifiers and function arrows. This is a lightweight approximation to concept-level search, effective for browsing the knowledge base by topic.

11.5 Subexpression and Pattern Index

A subexpression index maps each constant hash to the set of terms that mention it:

$$\mathsf{mentions}(h_c) = \{h \mid (h, t, T) \in \mathcal{S},\ h_c \text{ occurs in } t \text{ or } T\}$$

This supports queries like "find all terms that use Nat.add" or "find all proofs that apply Eq.trans." It is a form of full-text search for formalized mathematics.

A more expressive variant allows *pattern matching*: given a template expression with wildcards, find all terms containing a subexpression matching the pattern. For example, the pattern Nat.add ? 0 would find all terms that add zero to something. Efficient pattern matching over a large term corpus connects to the literature on code search and anti-unification.

11.6 Typeclass and Structure Instance Index

In systems like Lean 4, typeclass instances are key organizational structures. A typeclass instance index maps each typeclass (identified by the hash of its inductive declaration) to the set of instances registered for it:

$$\mathsf{instances}(h_{\mathsf{class}}) = \{h \mid (h, t, T) \in \mathcal{S},\ T \text{ has head constant } h_{\mathsf{class}}\}$$

This enables queries like "which types have a Group instance?" or "find all Monad implementations." Combined with the head-symbol index, it supports rich algebraic hierarchy browsing.

11.7 Normalized Type Index

Section 7.13 noted that the type index uses syntactic type hashes, so definitionally equal types with different syntax produce different index entries. A normalized type index addresses this by reducing types to a canonical form before hashing:

$$\mathcal{I}_{\mathsf{norm}}(h) = \mathcal{I}(\mathcal{H}(\mathsf{normalize}(T))) \quad \text{where} \quad \mathcal{H}(T) = h$$

The challenge is choosing a normalization strategy. Full $\beta\delta\iota\zeta$-normalization is computationally expensive (potentially exponential), but weaker normalizations—such as head-normalizing only the outermost structure, or normalizing with bounded unfolding depth—may capture most practical equivalences at acceptable cost.

11.8 Compositional Search

The indices above can be composed to answer complex queries. For example:

- "Find all constructive proofs of commutativity for rings": unification-based type search for $\forall(R : \mathsf{Ring}),\ ?a \cdot ?b = ?b \cdot ?a$, filtered by the axiom index to exclude classical axioms.
- "Find all lemmas about List.map that don't depend on Decidable": subexpression search for List.map, filtered by reverse-dependency exclusion.
- "Find all instances of Functor for types defined in terms of List": typeclass instance index for Functor, filtered by dependency index for List.

A query language that supports Boolean combinations of index predicates, analogous to database query planning, would make the knowledge base into a genuine search engine for formalized mathematics.

12 Conclusion

The Calculus of Inductive Constructions is the right foundation for a global mathematical knowledge base: it is the shared core of the most successful proof assistants, it is expressive enough for serious mathematics, and its type checker is small enough to be formally verified.

Content-addressing eliminates the coordination bottleneck that limits the scale of formalized mathematics. No naming consensus, no centralized curation, no review queues—just verified terms, identified by their structure, composable by hash.

The type-to-inhabitants index turns the knowledge base into a searchable library where AI agents and human mathematicians discover relevant results by type signature, not by remembering names.

HashMath provides the infrastructure for a future in which thousands of AI agents and hundreds of mathematicians contribute simultaneously, building on each other's work without a single naming conflict or a single gatekeeper.

Mathematics should be permissionless. HashMath makes it so.

References

[1] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. CADE-28*, 2021, pp. 625–635.

[2] The Mathlib Community, "Mathlib4," https://github.com/leanprover-community/mathlib4, 2024.

[3] AlphaProof and AlphaGeometry Teams, "AI achieves silver-medal standard solving International Mathematical Olympiad problems," *Nature*, 2025.

[4] Z. Ren et al., "DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition," *arXiv:2504.21801*, 2025.

[5] T. Tao, "Machine-assisted proofs," *ICM Invited Lecture*, 2025.

[6] W. T. Gowers and M. Nielsen, "Massively collaborative mathematics," *Nature*, vol. 461, pp. 879–881, 2009.

[7] D. H. J. Polymath, "A new proof of the density Hales-Jewett theorem," *Annals of Mathematics*, vol. 175, no. 3, pp. 1283–1327, 2012.

[8] T. Coquand and G. Huet, "The Calculus of Constructions," *Information and Computation*, vol. 76, pp. 95–120, 1988.

[9] T. Coquand and C. Paulin-Mohring, "Inductively defined types," in *Proc. COLOG-88*, LNCS 417, Springer, 1990.

[10] B. Werner, "Une théorie des constructions inductives," Ph.D. thesis, Université Paris 7, 1994.

[11] C. Paulin-Mohring, "Inductive definitions in the system Coq: Rules and properties," in *Proc. TLCA*, LNCS 664, Springer, 1993.

[12] L. Diehl, "Dependent types in 200 lines of Python," blog post, 2023.

[13] L. Diehl, "The Typechecker Zoo," https://github.com/larrytheliquid/typecheckerzoo, 2023.

[14] P. Chiusano and R. Bjarnason, "Unison: A friendly programming language from the future," https://www.unison-lang.org/, 2024.

[15] R. C. Merkle, "A certified digital signature," in *Proc. CRYPTO'89*, LNCS 435, Springer, 1990. (Original concept, 1979.)

[16] NIST, "Secure Hash Standard (SHS)," *FIPS PUB 180-4*, 2015.

[17] N. G. de Bruijn, "The mathematical language AUTOMATH, its usage, and some of its extensions," in *Symposium on Automatic Demonstration*, LNCS 125, Springer, 1970.

[18] R. Pollack, "How to believe a machine-checked proof," in *Twenty-Five Years of Constructive Type Theory*, Oxford, 1998.

[19] M. Sozeau, S. Boulier, Y. Forster, N. Tabareau, and T. Winterhalter, "Coq Coq correct! Verification of type checking and erasure for Coq, in Coq," *Proc. ACM Program. Lang.*, vol. 4 (POPL), pp. 8:1–8:28, 2020.

[20] G. Dowek, "Logipedia: a multi-system encyclopedia of formal proofs," *arXiv:2305.00064*, 2023.

[21] A. Assaf, G. Burel, R. Cauderlier, D. Delahaye, G. Dowek, C. Dubois, F. Gilbert, P. Halmagrand, O. Hermant, and R. Saillard, "Dedukti: a logical framework based on the $\lambda\Pi$-calculus modulo theory," 2016.

[22] F. Blanqui et al., "EuroProofNet: European research network on digital proofs," COST Action CA20111, 2022–2025.

[23] J. Benet, "IPFS—Content addressed, versioned, P2P file system," *arXiv:1407.3561*, 2014.

[24] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[25] P. Graf, "Term Indexing," *Lecture Notes in Computer Science*, vol. 1053, Springer, 1996.

[26] Protocol Labs, "libp2p: A modular network stack," https://libp2p.io/, 2024.

[27] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in *Proc. IPTPS*, LNCS 2429, Springer, 2002, pp. 53–65.